

cumacAiranSubcontext

Using cuMAC AI-RAN trtEngine inference in the cuPHY + cuMAC GPU-sharing test bench (**cubb_gpu_test_bench**)

Generated 2026-06-10

1. Overview

cumacAiranSubcontext is the inference-side analogue of **cumac::cumacSubcontext**. It owns one TensorRT (**trtEngine**) model-inference workload — the engine plus its host/device I/O buffers — and exposes the same **constructor / setup() / run()** lifecycle. That lets the existing cuMAC test worker drive model inference on a dedicated CUDA stream / (green) context instead of the cuMAC scheduler kernels. Running it next to the cuPHY workers is what exercises GPU sharing between cuPHY and cuMAC AI-RAN inference.

Like the scheduler subcontext, it is configured entirely from an HDF5 test vector (TV), so the worker only needs a file name. The worker authors that TV on the fly from YAML options using the bundled helper **cumac_ml::createAiranTv()**.

2. Components added

- **cuMAC/examples/ml/cumacAiranSubcontext.h / .cpp** — the inference subcontext class.
- **cuMAC/examples/ml/airanTvUtil.h / .cpp** — AI-RAN TV create/load helpers (**createAiranTv**, **loadAiranTv**, **airanTvParamsFromInfConfig**).
- **cuMAC/examples/ml/cumacAiranSubcontext/** (main.cpp + CMakeLists.txt) — standalone unit test.
- **testBenches/cubb_gpu_test_bench/** — opt-in worker integration (**cumac_testWrkr.hpp/.cpp**), YAML parsing (**cubb_gpu_test_bench.cpp**), and CMake wiring.

3. Prerequisites

- TensorRT runtime + parser (**libnvinfer.so**, **libnvonnxparser.so**) and the HDF5 C++ library.
- A trained model file: **.onnx** or a pre-serialized **.engine/trt/.plan** (recommended), or a **.pt** checkpoint (requires PyTorch the first time, then cached).
- Build the test bench inside the Aerial container/toolchain where the above are available.

4. Build the test bench with AI-RAN enabled

AI-RAN inference is **opt-in** and OFF by default, so existing builds are unaffected. Enable it with **-DENABLE_CUMAC_AIRAN=ON** (which also pulls in the trtEngine sources and links TensorRT). It requires **-DENABLE_CUMAC=ON**.

```
cmake -S <aerial_sdk> -B build \  
  -DENABLE_CUMAC=ON \  
  -DENABLE_CUMAC_AIRAN=ON  
cmake --build build --target cubb_gpu_test_bench
```

If **airan.infer=1** is set in YAML but the binary was built without **ENABLE_CUMAC_AIRAN**, the worker stops with a clear error.

5. Configure the YAML

Add an **airan:** block inside the existing **cumac_options:** section of your test bench launch config (e.g. testBenches/perf/cubb_gpu_test_config.yaml). It may live at the YAML root or under a **config:** parent.

```
cumac_options:
  half_precision: 0          # 0 fp32, 1 fp16 (also selects engine precision)
  sch_alg: 1
  airan:
    infer: 1                 # 1 = run trtEngine inference instead of scheduler
    model_path: /abs/path/model.onnx # .onnx / .engine / .pt
    input_name: obs
    output_name: logits
    obs_dim: 512             # model input feature dimension
    action_dim: 32           # model output dimension
    batch_size: 20           # active inference batch (UEs per forward pass)
    max_batch_size: 24       # dynamic-profile max (>= batch_size)
    use_cuda_graph: 1
    builder_opt_level: 5
```

6. Run the GPU-sharing test

Enable the cuMAC (MAC) context with **--T** and use the long pattern **-u 5**, passing your YAML config. The cuPHY channels and the cuMAC inference pipes then share the GPU through the same green-context / stream machinery.

```
./cubb_gpu_test_bench -u 5 --T -i cubb_gpu_test_config.yaml
```

Important: the MAC *slot timeline* is still driven by the presence of MAC test vectors in the YAML pattern (tdd_slot_config.MAC). In AI-RAN mode the inference pipes *replace* the scheduler in those slots but ignore the scheduler TV contents — you still list MAC TVs to allocate the slots.

7. cumac_options.airan reference

Key	Default	Meaning
infer	0	0 = cuMAC scheduler pipes, 1 = trtEngine inference pipes.
model_path	(empty)	Model file. .onnx / .engine .trt .plan / .pt (auto-detected). Required when infer=1.
input_name	obs	Model graph input tensor name.
output_name	logits	Model graph output tensor name.
obs_dim	487	Input feature dimension.
action_dim	28	Output dimension.
batch_size	16	Active per-inference batch size bound at runtime.
max_batch_size	16	Max batch of the TensorRT dynamic optimization profile (clamped up to batch_size).
use_cuda_graph	1	Capture + replay the enqueue with a CUDA Graph (lower launch latency).
builder_opt_level	5	TensorRT builder optimization level (0-5; <0 = TRT default).

Note: precision is taken from **cumac_options.half_precision** (0 = fp32, 1 = fp16); a value of 1 forces an FP16 engine.

8. Standalone unit test

The **cumacAiranSubcontext** unit test exercises the subcontext through the exact init / setup / run / eval lifecycle the worker uses, then validates the output and reports per-inference latency — confirming it works as expected

before running the full bench.

```
# build
cmake --build build-ml --target cumacAiranSubcontext

# run (resolves the bundled infConfig.yaml when -c is omitted)
./cumacAiranSubcontext -c <infConfig.yaml> -g 0 -s 4
```

Options: **-c** infConfig.yaml (model + dims), **-t** output TV path, **-g** GPU index, **-s** number of MAC slots / pipes, **-n** timing iterations. Pointing **-c** at **trtEngine/generatedModels/mlp_4x512_relu.infConfig.yaml** reuses the pre-generated .onnx cache, so no PyTorch is needed.

9. How it works

The cuMAC test worker maps its per-slot lifecycle onto the inference pipes:

- **macInit** — writes a per-worker TV (airan_tv_<worker>.h5) from the YAML options and constructs one cumacAiranSubcontext pipe per MAC slot.
- **macSetup** — binds each pipe's engine I/O for the active batch and primes it once (this triggers CUDA Graph capture when enabled).
- **macRun** — for each slot, records a start event, runs inference on the stream, and records an end event — the same per-slot timing used for the scheduler.
- **eval** — per-slot GPU time is computed from a common start event.

GPU sharing uses the unchanged green-context / MPS / stream partitioning of the test bench; the inference engine is built once and cached on disk (keyed by precision, batch and optimization level), so additional pipes load the cached engine.

10. Notes & troubleshooting

- **batch_size vs max_batch_size**: changing batch_size within [1, max_batch_size] reuses the cached engine; exceeding max_batch_size rebuilds a separate engine.
- Prefer an absolute **model_path**; relative paths resolve against the process working directory.
- The per-worker TV is written to the working directory as airan_tv_<workerName>.h5.
- AI-RAN mode disables the cuMAC CPU reference check (it does not apply to inference).